

HOW SECURITY DEFINER EXECUTES INTERNALLY?

Code Link: <https://github.com/postgres/postgres/blob/master/src/backend/utils/fmgr/fmgr.c>

STEP 1: `pg_proc` (Function Catalog)

`pg_proc` = PostgreSQL's internal table that stores metadata about every function

When you do:

```
CREATE FUNCTION test(a int) RETURNS int ...
```

Postgres does **NOT** just save the code.

It creates a row in `pg_proc`:

```
Function name  
Who owns it  
How many args  
Return type  
Language  
SECURITY DEFINER or not  
Source code / implementation
```

So later, when someone calls:

```
SELECT test(10);
```

Postgres does:

```
Look in pg_proc  
Read metadata  
Decide how to run function
```

`pg_proc` is declared in this file: `src/include/catalog/pg_proc.h`

⇒ Important fields (only ones we care about now)

(a) `oid`

```
Oid oid;
```

Function ID.

Example:

```
sum(int,int) → OID 2108  
myfunc() → OID 50000
```

Postgres internally uses OID, not function name.

(b) `proname`

```
NameData proname;
```

Function name: Human-readable only.

Internally OID matters more.

(c) `proowner`

This is critical for SECURITY DEFINER.

```
Oid proowner;
```

Means:

```
Who owns this function?
```

Example:

```
CREATEFUNCTION ...
```

If created by `postgres`:

```
proowner = OID of postgres role
```

Later SECURITY DEFINER uses this.

(d) `prosecdef`

Most important field for your work.

```
bool prosecdef;
```

Means:

```
Is this SECURITY DEFINER?
```

Values:

```
false → SECURITY INVOKER (default)  
true  → SECURITY DEFINER
```

(e) `prolang`

```
Oid prolang;
```

Which language?

(f) `prosrc`

```
text prosrc;
```

Function source.

(g) `pronargs`

```
int16 pronargs;
```

How many arguments.

Key takeaway:

The **two fields SECURITY DEFINER** depends on are:

```
prosecdef → should wrapper be used?  
proowner → which role should wrapper switch to?
```

STEP 2: Function lookup (**fmgr_info**)

What happens when user calls a function?

Suppose user runs:

```
SELECT check_customer(10);
```

Internally PostgreSQL does NOT immediately execute code. Flow:

```
SQL parser resolves function name  
↓  
finds function OID  
↓  
calls fmgr_info(functionId, &finfo)  
↓  
reads pg_proc  
↓  
builds execution metadata
```

So **fmgr_info()** is the gateway to function execution.

Entry point in source

In **fmgr.c**: (Line no **128**)

```
void  
fmgr_info(OidfunctionId, FmgrInfo*finfo)  
{
```

```
fmgr_info_cxt_security(functionId, finfo, CurrentMemoryContext,
false);
}
```

This is a wrapper. It simply forwards to:

```
fmgr_info_cxt_security(...)
```

That function does the real work.

This is the main code: Line no 149

```
static void
fmgr_info_cxt_security(Oid functionId, FmgrInfo *finfo, MemoryContext mcxt,
                        bool ignore_security)
{
    const FmgrBuiltin *fbp;
    HeapTuple    procedureTuple;
    Form_pg_proc procedureStruct;
    Datum        prosrcdatum;
    char         *prosrc;

    /*
     * fn_oid *must* be filled in last. Some code assumes that if fn_oid is
     * valid, the whole struct is valid. Some FmgrInfo structs do survive
     * elogs.
     */
    finfo->fn_oid = InvalidOid;
    finfo->fn_extra = NULL;
    finfo->fn_mcxt = mcxt;
    finfo->fn_expr = NULL; /* caller may set this later
    */
```

```

if ((fbp = fmgr_isbuiltin(functionId)) != NULL)
{
    /*
     * Fast path for builtin functions: don't bother cons
ulting pg_proc
     */
    finfo->fn_nargs = fbp->nargs;
    finfo->fn_strict = fbp->strict;
    finfo->fn_retset = fbp->retset;
    finfo->fn_stats = TRACK_FUNC_ALL; /* ie, never trac
k */
    finfo->fn_addr = fbp->func;
    finfo->fn_oid = functionId;
    return;
}

/* Otherwise we need the pg_proc entry */
procedureTuple = SearchSysCache1(PROC0ID, ObjectIdGetDatu
m(functionId));
if (!HeapTupleIsValid(procedureTuple))
    elog(ERROR, "cache lookup failed for function %u", fu
nctionId);
procedureStruct = (Form_pg_proc) GETSTRUCT(procedureTupl
e);

finfo->fn_nargs = procedureStruct->pronargs;
finfo->fn_strict = procedureStruct->proisstrict;
finfo->fn_retset = procedureStruct->proretset;

/*
 * If it has proconfig set, non-null proconfig, or if a p
lugin wants to
 * hook function entry/exit, use fmgr_security_definer ca
ll handler ---
 * unless we are being called again by fmgr_security_defi
ner or

```

```

    * fmgr_info_other_lang.
    *
    * When using fmgr_security_definer, function stats tracking
    is always
    * disabled at the outer level, and instead we set the flag
    properly in
    * fmgr_security_definer's private flinfo and implement the
    tracking
    * inside fmgr_security_definer. This loses the ability to
    charge the
    * overhead of fmgr_security_definer to the function, but
    gains the
    * ability to set the track_functions GUC as a local GUC
    parameter of an
    * interesting function and have the right things happen.
    */
    if (!ignore_security &&
        (procedureStruct->prosecdef ||
         !heap_attisnull(procedureTuple, Anum_pg_proc_proconf
ig, NULL) ||
         FmgrHookIsNeeded(functionId)))
    {
        finfo->fn_addr = fmgr_security_definer;
        finfo->fn_stats = TRACK_FUNC_ALL; /* ie, never trac
k */
        finfo->fn_oid = functionId;
        ReleaseSysCache(procedureTuple);
        return;
    }

    switch (procedureStruct->prolang)
    {
        case INTERNALlanguageId:

            /*
             * For an ordinary builtin function, we should ne

```

```

ver get here
    * because the fmgr_isbuiltin() search above will
have succeeded.
    * However, if the user has done a CREATE FUNCTIO
N to create an
    * alias for a builtin function, we can end up he
re. In that case
    * we have to look up the function by name. The
name of the
    * internal function is stored in prosrc (it does
n't have to be
    * the same as the name of the alias!)
    */
prosrcdatum = SysCacheGetAttrNotNull(PROC0ID, pro
cedureTuple,
                                Anum_pg_proc
_prosrc);
prosrc = TextDatumGetCString(prosrcdatum);
fbp = fmgr_lookupByName(prosrc);
if (fbp == NULL)
    ereport(ERROR,
            (errcode(ERRCODE_UNDEFINED_FUNCTION),
             errmsg("internal function \"%s\" is
not in internal lookup table",
                    prosrc)));
pfree(prosrc);
/* Should we check that nargs, strict, reset mat
ch the table? */
finfo->fn_addr = fbp->func;
/* note this policy is also assumed in fast path
above */
finfo->fn_stats = TRACK_FUNC_ALL; /* ie, never
track */
break;

case ClanguageId:

```

```

        fmgr_info_C_lang(functionId, finfo, procedureTupl
e);
        finfo->fn_stats = TRACK_FUNC_PL;    /* ie, track
if ALL */
        break;

        case SQLlanguageId:
            finfo->fn_addr = fmgr_sql;
            finfo->fn_stats = TRACK_FUNC_PL;    /* ie, track
if ALL */
            break;

        default:
            fmgr_info_other_lang(functionId, finfo, procedure
Tuple);
            finfo->fn_stats = TRACK_FUNC_OFF;    /* ie, track
if not OFF */
            break;
    }

    finfo->fn_oid = functionId;
    ReleaseSysCache(procedureTuple);
}

```

What is `SearchSysCache1()` ?

```

procedureTuple = SearchSysCache1(PROC0ID, ObjectIdGetDatum(fu
nctionId));

```

Conceptually: Search pg_proc using OID. Equivalent SQL idea:

```

SELECT * FROM pg_proc WHERE oid=50001;

```

But Postgres does not use SQL here — it uses system cache.

Breakdown

`PROCoid` : Use `pg_proc` catalog cache

`functionId` : Find row for this function

Result Returns: `HeapTuple procedureTuple` ;

`HeapTuple` = `one raw database row` but not decoded yet.

Conceptually contains:

```
oid = 50001
prname = check_customer
proowner = postgres
prosecdef = true
...
```

But still raw memory.

Converting raw row into struct

```
procedureStruct = (Form_pg_proc) GETSTRUCT(procedureTuple);
```

Now PostgreSQL can do:

```
procedureStruct->proowner
procedureStruct->prosecdef
procedureStruct->prolang
```

What is `Form_pg_proc` ?

This comes from `pg_proc.h`.

Conceptually:

```
typedef FormData_pg_proc *Form_pg_proc;
```

Meaning: pointer to catalog row struct.

So this: `procedureStruct->prosecdef`

Why `FmgrInfo` exists ?

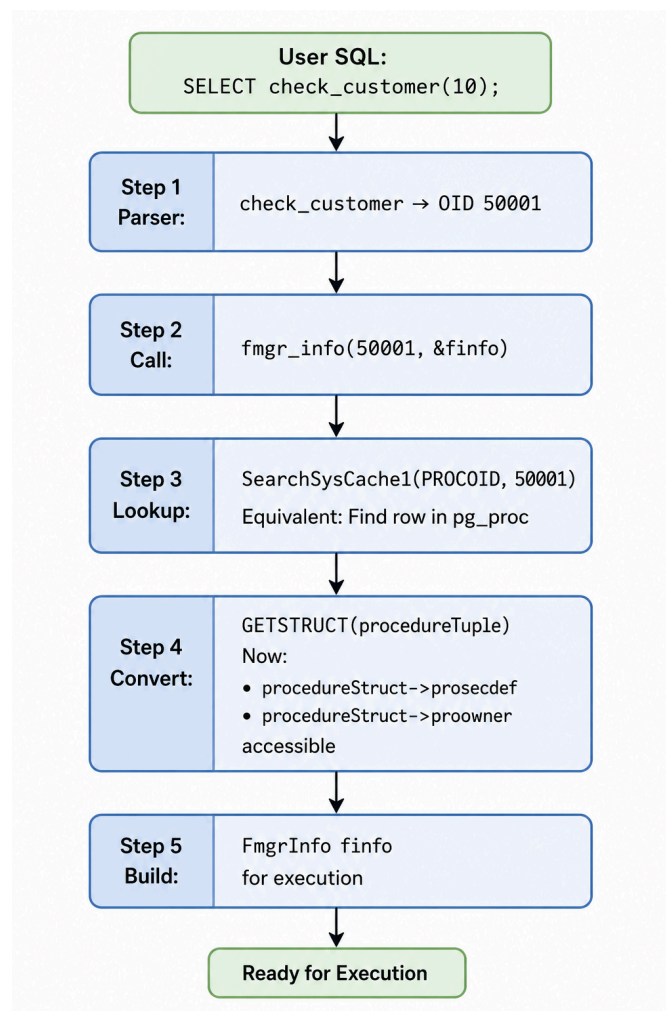
`FmgrInfo` is runtime execution info. Think: `pg_proc`

Permanent metadata:

stored in catalog

`FmgrInfo` Runtime copy:

used for actual function execution



STEP 3: SECURITY DEFINER detection

Inside `fmgr_info_cxt_security()`:

```

if (!ignore_security && (procedureStruct->prosecdef || !heap_attisnull(...proconfig ...) || FmgrHookIsNeeded(functionId))) {
    finfo->fn_addr=fmgr_security_definer;
    ...
    return;
}

```

This is the decision point. Simplified:

```

if (procedureStruct->prosecdef) // checking SECURITY DEFINER
    true or false
{
    finfo->fn_addr=fmgr_security_definer;
}

```

```

if (!ignore_security &&
    (procedureStruct->prosecdef ||
     !heap_attisnull(procedureTuple, Anum_pg_proc_proconfig, NULL) ||
     FmgrHookIsNeeded(functionId)))
{
    finfo->fn_addr = fmgr_security_definer;
    finfo->fn_stats = TRACK_FUNC_ALL; /* ie, never track */
    finfo->fn_oid = functionId;
    ReleaseSysCache(procedureTuple);
    return;
}

```

```

finfo->fn_addr = fmgr_security_definer;

```

PostgreSQL is basically doing:

Instead of: Call original function

it changes to: Call wrapper

Wrapper decides what happens.

Why `return` is important here?

Inside:

```
finfo->fn_addr=fmgr_security_definer;  
return;
```

After wrapper assignment: Postgres stops. It does NOT continue to:

```
loadSQLExecutor  
loadCexecutor  
loadPLpgSQLhandler
```

Because wrapper will do that later.

Key takeaway of STEP 3:

This line is the entire SECURITY DEFINER trigger:

```
finfo->fn_addr=fmgr_security_definer;
```

It means:

```
Do NOT call real function directly  
Call security wrapper first
```

That is PostgreSQL's core design.

```
finfo->fn_addr=fmgr_security_definer;
```

How does PostgreSQL actually call that? Answer: through `FmgrInfo` .

What is `FmgrInfo` ?

Short answer: `FmgrInfo` = runtime execution structure for a function

Structure of `FmgrInfo`

Conceptually (simplified):

```
typedef struct FmgrInfo
{
    PGFunction fn_addr; // code to execute
    Oid fn_oid; // function id
    short fn_nargs; // number of args
    bool fn_strict;
    bool fn_retset;
    void *fn_extra;
    MemoryContext fn_mcxt;
    Node *fn_expr;
} FmgrInfo;
```

Real struct has some more details, but this is enough.

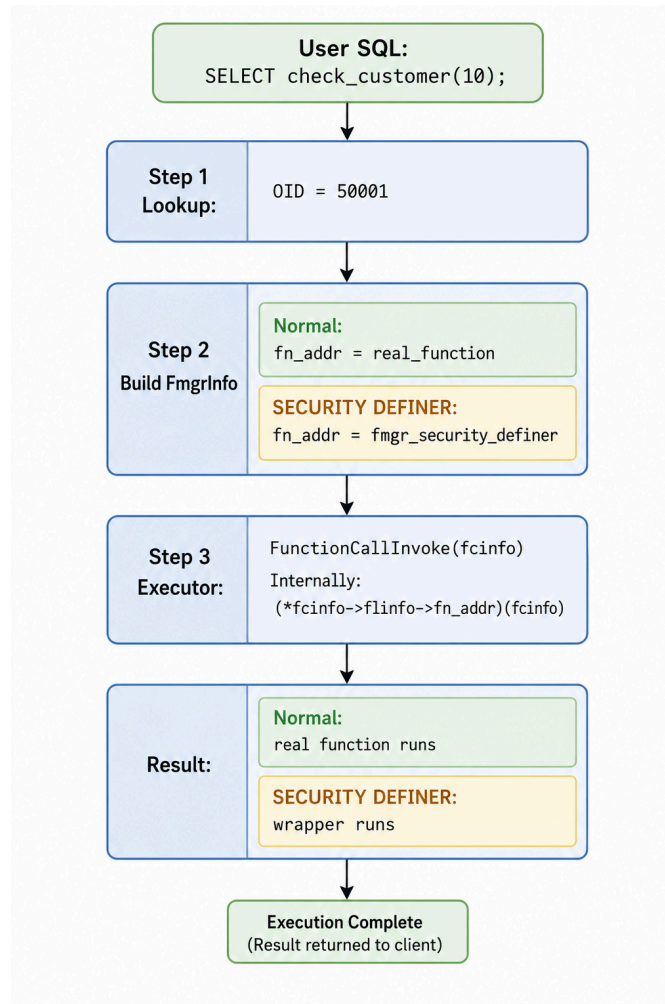
What is `PGFunction`?

Internally:

```
typedef Datum (*PGFunction)(FunctionCallInfo fcinfo);
```

This means:

`PGFunction` = pointer to executable function



STEP 4: Inside `fmgr_security_definer()` Line No 663

```
fcinfo->fn_addr = fmgr_security_definer;
```

Function entry Real function:

```
Datum fmgr_security_definer(PG_FUNCTION_ARGS)
{
    ...
}
```

This is the wrapper that runs instead of real function.

Important local variables

At top:

```
Datum result;  
struct fmgr_security_definer_cache *fcache;  
FmgrInfo *save_flinfo;  
Oid save_userid;  
int save_sec_context;
```

These will be used for:

Variable	Purpose
result	store function return value
fcache	private cache for wrapper
save_userid	original caller
save_sec_context	original security state
save_flinfo	save old function info

So: if (!fcinfo→flinfo→fn_extra)

means:

```
Do I already have cached wrapper data?
```

Cache struct Defined above:

```
struct fmgr_security_definer_cache  
{  
    FmgrInfo flinfo;  
    Oid userid;  
    List *configNames;  
    List *configValues;  
    Datum arg;  
};
```

This cache stores:

Field	Meaning
<code>flinfo</code>	real underlying function
<code>userid</code>	owner role
<code>config...</code>	GUC configs
<code>arg</code>	plugin support

For SECURITY DEFINER, the critical one is: Oid userid;

This becomes:

```
role to switch into
```

Why does wrapper call `fmgr_info()` again?

Remember current situation: Outer `FmgrInfo`:

```
fn_addr = fmgr_security_definer
```

That is wrapper. But wrapper now needs:

```
real underlying function
```

So it re-runs lookup.

Why `ignore_security = true`? Call:

```
fmgr_info_cxt_security(...,true)
```

That `true` means:

```
Ignore SECURITY DEFINER detection
```

Why? If not: wrapper lookup would do:

```
see prosexdef = true
↓
assign fn_addr = fmgr_security_definer again
↓
wrapper calls wrapper
↓
wrapper calls wrapper
↓
infinite recursion
```

So Postgres says:

This time give me the REAL function not the wrapper

Result after this line `fcache->flinfo` now contains:

Instead of:

```
fn_addr = fmgr_security_definer
```

it contains:

```
fn_addr = actual SQL/C/PLpgSQL function
```

Now wrapper has two things: Outer `fcinfo->flinfo`

```
wrapper info
fn_addr = fmgr_security_definer
```

Inner `fcache->flinfo`

```
real function info
fn_addr = actual function
```

Key takeaway of Section 5

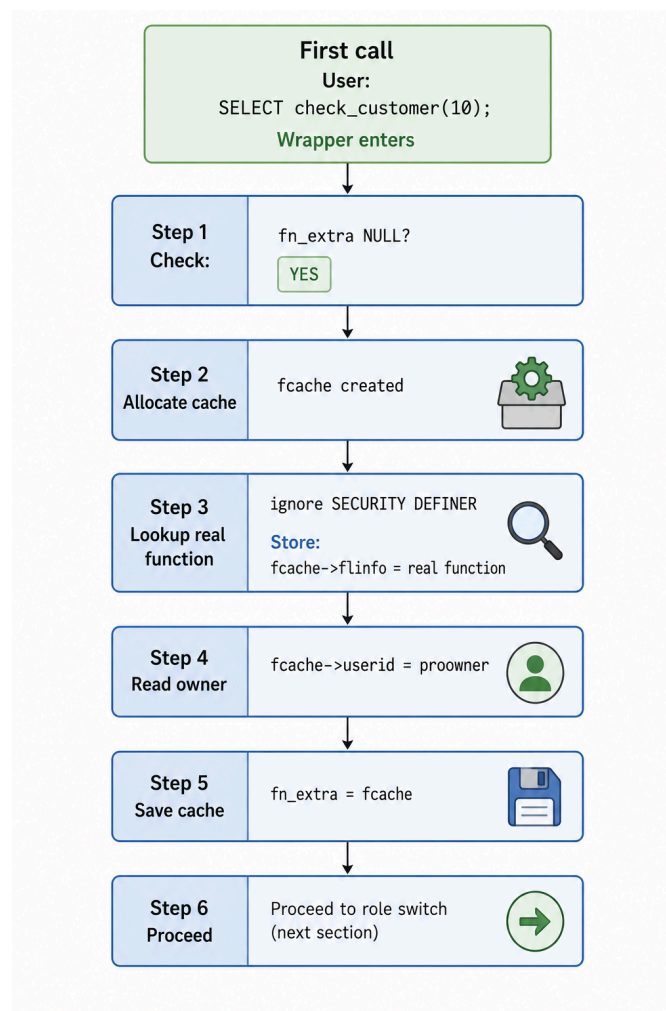
Wrapper builds a private cache containing **two critical things**:

`fcache->flinfo`

real underlying function

`fcache->userid`

function owner



STEP 5: Role switch

(`GetUserIdAndSecContext`) /

SetUserIdAndSecContext)

Postgres does:

```
GetUserIdAndSecContext(  
    &save_userid,  
    &save_sec_context  
);
```

This is the first half of role switching.

What does this do? Think:

```
Save current security state
```

Before: Current execution:

```
current_user = alice  
security flags = normal
```

After: Variables contain:

```
save_userid = alice_oid  
save_sec_context = current flags
```

Why save first? Because PostgreSQL must restore later. Without saving:

```
switch to postgres  
run function  
who do we restore to?  
unknown
```

Security disaster. So think:

```
save before changing
```

Second important line This is the real switch:

```
if (OidIsValid(fcache->userid))  
    SetUserIdAndSecContext(  
        fcache->userid,  
        save_sec_context | SECURITY_LOCAL_USERID_CHANGE  
    );
```

This is the exact SECURITY DEFINER implementation.

OidIsValid(fcache->userid) ⇒ Do we have a valid owner role?

SetUserIdAndSecContext(...) ⇒ Change current execution user

fcache->userid ⇒ function owner

SetUserIdAndSecContext(postgres, ...) ⇒ CurrentUserId = postgres

Now every permission check sees:

```
postgres
```

not alice. THIS is the actual privilege escalation.

Why RLS changes here

This is where your cyclic policy issue connects. Suppose function does:

```
SELECT * FROM customers;
```

Before switch:

```
CurrentUserId = alice  
RLS applies
```

After switch:

```
CurrentUserId = postgres  
RLS bypass
```

That is why SECURITY DEFINER changes behavior.

```
alice calls function
```

```
↓
```

```
save alice
```

```
↓
```

```
switch to postgres
```

```
↓
```

```
function runs
```

```
↓
```

```
restore alice
```

- Wrapper built cache
- Saved caller
- Switched role (`alice → postgres`)

STEP 6: Actual function execution inside wrapper

LINE NO `740`.

Where we are in code After role switch, wrapper reaches this:

```
save_flnfo=fcinfo->flinfo;
```

```
PG_TRY();
```

```
{
```

```
    fcinfo->flinfo=&fcache->flinfo;
```

```
    result=FunctionCallInvoke(fcinfo);
```

```
}
```

This is the real function call logic.

Current state: Outer call came from:

```
FunctionCallInvoke(fcinfo)
```

At that moment:

```
fcinfo->flinfo = wrapper FmgrInfo  
fn_addr = fmgr_security_definer
```

Meaning:

```
If wrapper calls FunctionCallInvoke(fcinfo) directly...  
it would call itself again
```

Infinite recursion.

Why would recursion happen?

Remember: `FunctionCallInvoke(fcinfo)` roughly means:

```
(*fcinfo->flinfo->fn_addr)(fcinfo)
```

Suppose wrapper did nothing: Current:

```
fcinfo->flinfo->fn_addr = fmgr_security_definer
```

Then:

```
FunctionCallInvoke(fcinfo)
```

becomes:

```
fmgr_security_definer(fcinfo)
```

So:

```
wrapper  
→ wrapper
```

- wrapper
- infinite recursion

That would crash.

PostgreSQL's solution: This line:

```
save_flnfo=fcinfo->flinfo;
```

Means:

```
Save current wrapper FmgrInfo
```

What is `fcache->flinfo` again?

Wrapper created:

```
fmgr_info_cxt_security(...,true)
```

That produced:

```
fcache->flinfo
```

Containing:

```
fn_addr = real SQL/C/PLpgSQL function
```

NOT wrapper.

So now: Outer `fcinfo->flinfo`

(original)

```
fn_addr = fmgr_security_definer
```

Wrapper replaces with:

```
fcache->flinfo
```

which contains:

```
fn_addr = real function
```

`FunctionCallInvoke(fcinfo) ⇒ (*fcinfo->flinfo->fn_addr)(fcinfo)`

`fcinfo->flinfo = real function info`

Function return value

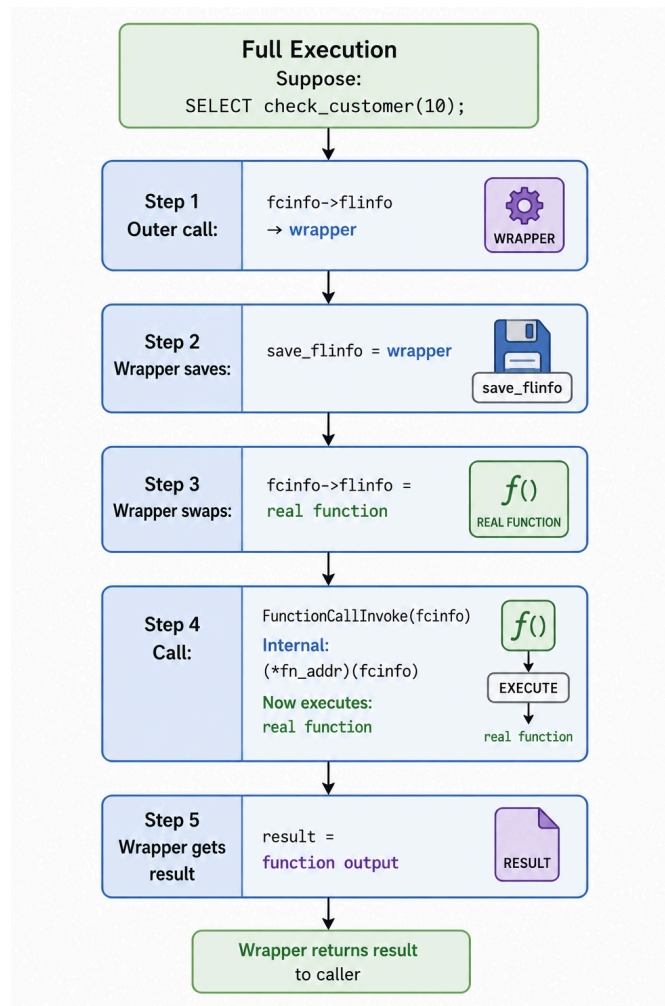
This line:

```
result=FunctionCallInvoke(fcinfo);
```

Stores:

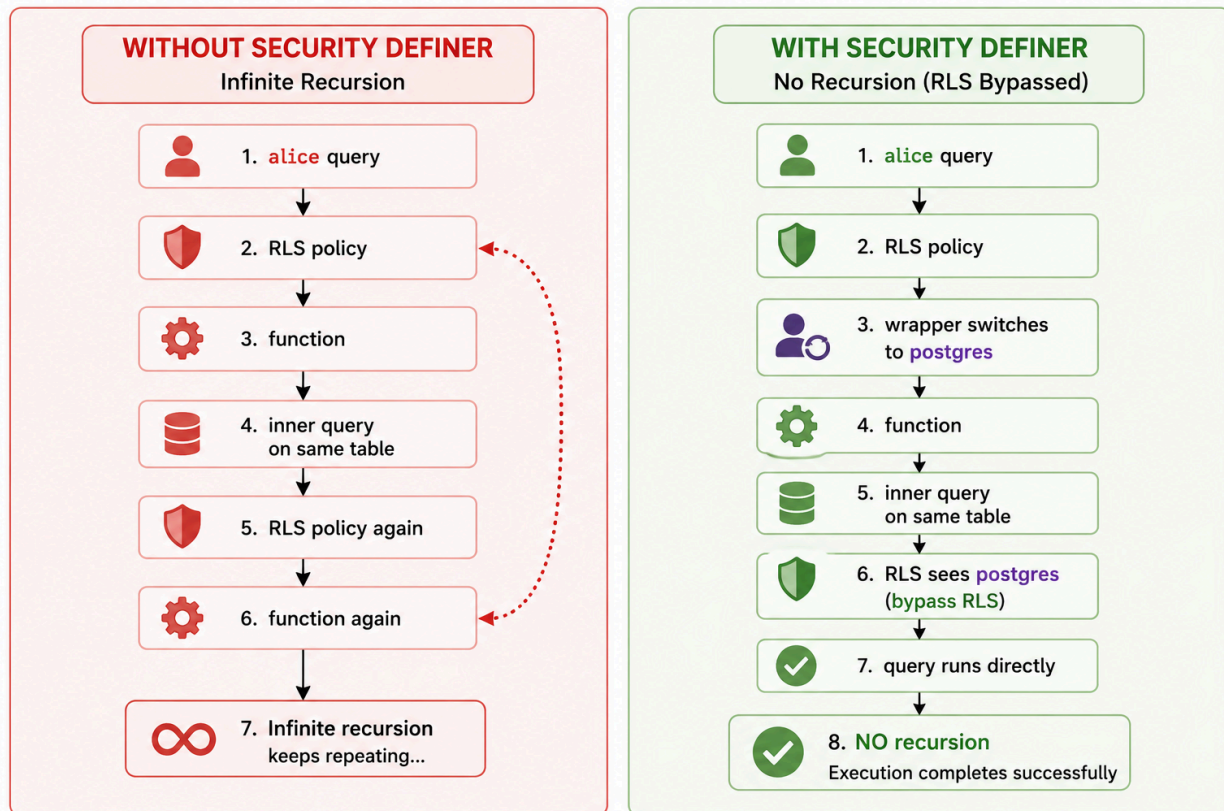
```
whatever real function returns
```

Later wrapper returns same result to caller. So caller doesn't even know wrapper existed.



```

alice calls function
↓
wrapper runs
↓
switches CurrentUserId → postgres
↓
real function runs
  
```



In PostgreSQL, RLS does not check who originally sent the query; it checks `CurrentUserId` (the user under which the code is executing right now). Normally, if `alice` runs a query, `CurrentUserId = alice`, so RLS policies are applied using Alice's permissions. But when a `SECURITY DEFINER` function is called, PostgreSQL's wrapper temporarily changes `CurrentUserId` from `alice` to the **function owner** (for example `postgres`) using `SetUserIdAndSecContext()`. Then, when the function executes an inner query on the same table, RLS checks see `CurrentUserId = postgres` instead of Alice. If that owner is a superuser or has `BYPASSRLS`, PostgreSQL skips RLS for that inner query, so the policy does not recursively call itself again, which is why cyclic RLS recursion stops. So the real mechanism is: **SECURITY DEFINER changes execution identity, and because RLS uses that identity for permission checks, RLS behavior changes automatically.**

PostgreSQL SECURITY DEFINER – Complete Flow (Sections 1 to 7)

